

Запускать так: `./run.sh`

Сперва было написано стандартное решение рюкзака на $O(nW)$ с алгосов 2 семестра. То есть заводилась таблица размера $n \times W$ и в $dp[i][j]$ указывалась максимальная стоимость, которую можно уместить в рюкзак размера W используя первые n объектов. Для такого алгоритма выполнена корректность в том смысле, что ответ действительно даёт максимальную стоимость. В реализации выделяется nW памяти под инты, после чего делается 2 пробежки по выделенному массиву, будем считать, что затрачивается $4nW$ бит и nW времени. Чтобы укладываться в лимиты, будем считать, что не хорошо выходить за 0.5 гб памяти, что значит $4nW \leq 2^{29} \Leftrightarrow nW \leq 2^{27}$, так как количество операций мы считаем nW , то этого же ограничения и хватит для работы за не слишком большое время, поэтому им и ограничимся. В самой первой реализации было написано за $O(nW)$, если $nW \leq 2^{29}$ и ничего не класть в рюкзак иначе. Такое решение прошло большие пороги на 1, 2, 3, 5 тестах, поскольку решение за nW находит точный ответ¹, то пойдя по такому пути мы дополнительно выводим что ответ точный, итого 24 балла 6,6,6,0,6,0.

Далее была написана смесь алгоритма на подобию Ибарра-Ким и решения через dp. Возьмём $c = \lceil \frac{nW}{2^{27}} \rceil$ и увеличим вес каждого груза так, чтобы он делился на c . Поделим всё на НОД, сделаем алгоритм за nW , найдём сколько места осталось в рюкзаке, всё оставшееся место дополним алгоритмом за nW' , W' — меньше чем W , так что так может получиться. Иначе предварительно повторим процедуру, выбрав для остатка c побольше, чтобы ускорять последующие шаги и уложиться во время. Этот подход дал пройти четвёртый тест на 5, итого 29: 6,6,6,5,6,0.

В этом решении мы сперва отказались от увеличения c бездумно: скорее всего мы что-то протолкнули в массив в первый раз, так что алгоритм будет работать быстрее и так. То есть сперва мы пытаемся не менять c . Если с одним c не вышло, то попробуем сперва увеличить c на 1, что изменит остатки и даст элементам протолкнуться по другому. Также заметим, что на первой итерации мы находим довольно грубое приближение. Так что разумно было бы брать не все такие объекты, а часть из них, а если остаток был оптимальный то мы найдём его после ещё одной итерации. Такая константа должна быть не слишком велика, чтобы не забить рюкзак неоптимальными решениями и при этом не слишком мала, поскольку рюкзак, хоть и округлённый, однако для конкретного округления видит как стыковать блоки, а при меньшем это решение потеряется. Также слишком маленькая константа может застопорить алгоритм.

Выбрав $\alpha = 0.25$ были пройдены все 6 тестов на максимум. Метрики составили

99798, 142156, 100236, 3967168, 109899, 1099883.

Что дало 34 балла так: 6, 6, 6, 5, 6, 5. Отныне будем пытаться оптимизировать память. На деле наше решение не столь плохо с ней и сейчас. Если заменить $nW \leq 2^{27}$ на $nW \leq 2^{20}$, то метрики составят

99798, 142156, 100236, 3967173, 109899, 1099128.

То есть на 1, 2, 3, 5 тестах будут получены точные ответы, на остальных ответы немного ухудшатся. При этом на таких значениях алгоритм ни на одном тесте не пользуется динамикой за nW в чистом виде, получая метрики 5, 5, 5, 5, 5, 0, в сумме 25, хоть мы и знаем, что 4 раза он нашёл настоящий оптимум.

¹Ну это мы наверное знаем, потому что сдали на прошедшем курсе алгосов